

Exactly Protocol

SMART CONTRACT

Security Audit

Performed on Contracts:

Market.sol

SHA Hash: 9bd8d2e29f5d9893d88e3a972e210b6ed06827bc

InstallmentsRouter.sol

SHA Hash: 52809e9255bbd82342b00da6af411169bcbdd73c

Github Commit Hash: 9bd8d2e29f5d9893d88e3a972e210b6ed06827bc

Platform

OPTIMISM

hashlock.com.au

APRIL 2024

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Audit Findings	12
Centralisation	16
Conclusion	17
Our Methodology	18
Disclaimers	20
About Hashlock	21

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Exactly protocol team partnered with Hashlock to conduct a security audit of their Market.sol and InstallmentsRouter.sol smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Exactly is a decentralized and open-source DeFi protocol that allows users to easily exchange the value of their crypto assets through deposits and borrows with variable and fixed interest rates.

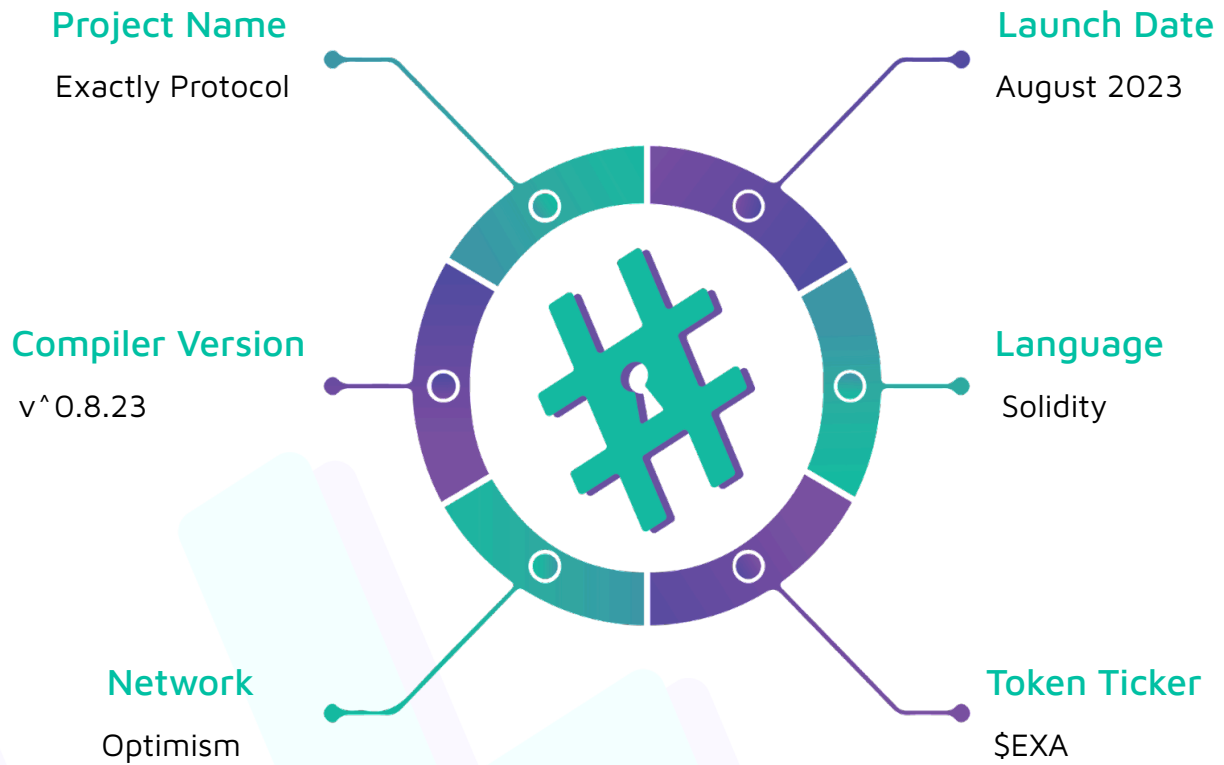
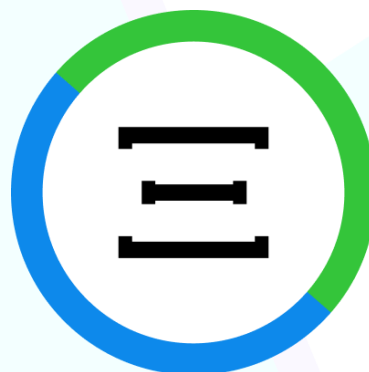
Project Name: Exactly Protocol

Compiler Version: ^0.8.23

Website: <https://exact.ly/>

Logo:

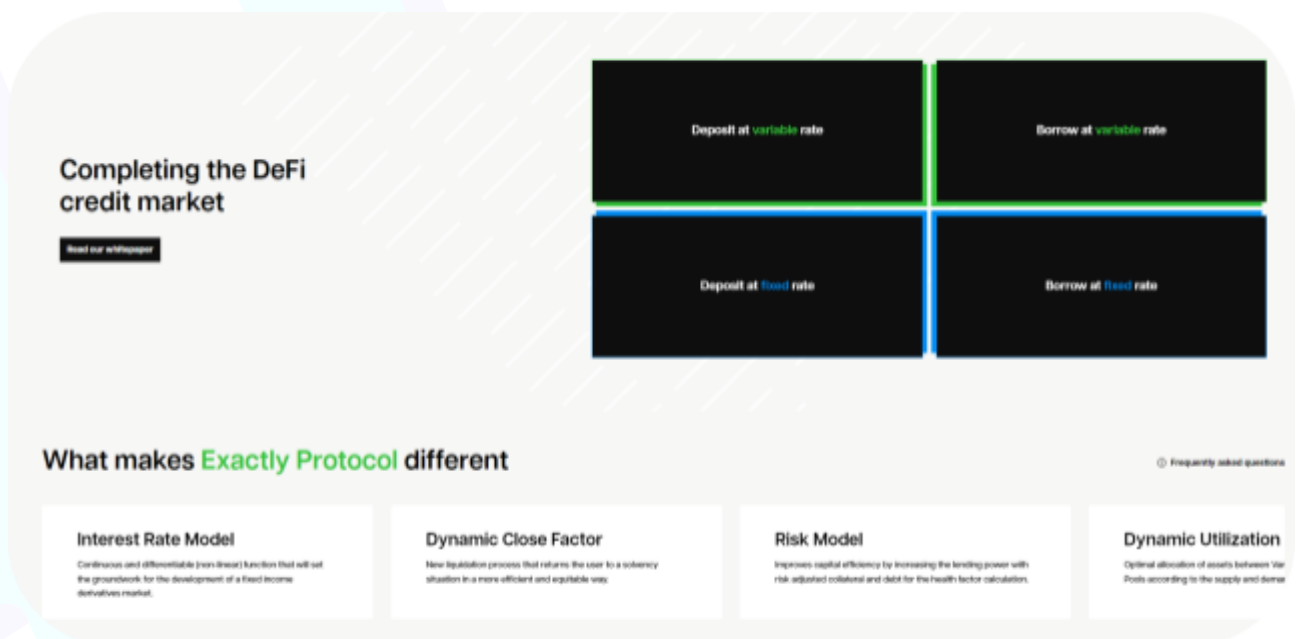
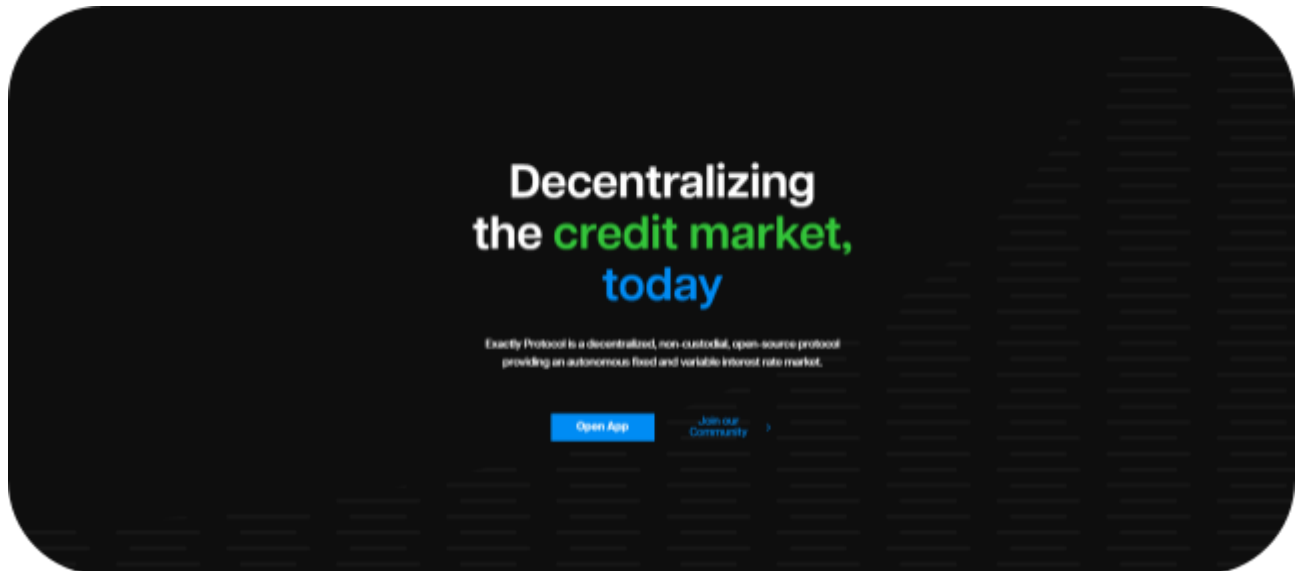
The logo for the Exactly Protocol, featuring a stylized 'E' symbol followed by the text 'exactly protocol' in a lowercase, sans-serif font.

Visualised Context:**Iconography:**

#Hashlock.

Hashlock Pty Ltd

Project Visuals:



#Hashlock.

Hashlock Pty Ltd

Audit scope

We at Hashlock audited the solidity code within the Exactly project, the scope of works included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line by line analysis and were supported by software assisted testing.

Description	Exactly Protocol Smart Contracts
Platform	Optimism Mainnet
Audit Date	April, 2024
Contract 1	Market.sol
Contract 1 SHA Hash	9bd8d2e29f5d9893d88e3a972e210b6ed06827bc
Contract 2	InstallmentsRouter.sol
Contract 2 SHA Hash	9bd8d2e29f5d9893d88e3a972e210b6ed06827bc
GitHub Commit Hash	9bd8d2e29f5d9893d88e3a972e210b6ed06827bc

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Hashlocked"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section.

Hashlock found:

0 High severity vulnerabilities

0 Medium severity vulnerabilities

1 Low severity vulnerabilities

3 Gas Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour
Market.sol - Main contract of the protocol. It exposes all user-oriented actions for fixed and variable borrows, deposits, repayments, and withdrawals.	Contract achieves this functionality.
InstallmentsRouter.sol - Facilitate multiple fixed borrow operations in a single transaction. It enables users to choose a specific market, the firstMaturity, the amounts to borrow, and a maxRepay for slippage.	Contract achieves this functionality.

Code Quality

This Audit scope involves the smart contracts of the Exactly project, as outlined in the Audit Scope section. All contracts, libraries and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however some refactoring is required.

The code is very well commented and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Exactly project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in understanding the overall architecture of the protocol.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues and inefficiencies

Audit Findings

Low

[L-01] InstallmentsRouter.sol#wrap - Unused modifier.

Description

There wrap modifier defined in InstallmentsRouter.sol contract is not used.

Recommendation

Remove the modifier as it is not used in the code.

```
modifier wrap() {  
  
    weth.deposit{ value: msg.value }();  
  
    -;  
  
}
```

Status

Resolved.

Gas

[G-01] Market.sol - Use assembly to check for address(0)

Description

There are some checks for address(0) that can be optimised using assembly.
Instances:

```
File: Market.sol
```

```
149:          if (address(memRewardsController) != address(0))
memRewardsController.handleBorrow(borrower);

210:          if (address(memRewardsController) != address(0))
memRewardsController.handleBorrow(borrower);

297:          if (address(memRewardsController) != address(0))
memRewardsController.handleBorrow(borrower);

492:          if (address(memRewardsController) != address(0))
memRewardsController.handleBorrow(borrower);

636:          if (address(memRewardsController) != address(0))
memRewardsController.handleBorrow(borrower);

686:          if (address(memRewardsController) != address(0))
memRewardsController.handleDeposit(borrower);

729:          if (address(memRewardsController) != address(0))
memRewardsController.handleDeposit(owner);

743:          if (address(memRewardsController) != address(0))
memRewardsController.handleDeposit(owner);

750:          if (address(memRewardsController) != address(0))
memRewardsController.handleDeposit(to);

761:  if (address(memRewardsController) != address(0)) {
```

```

776:    if (address(memRewardsController) != address(0)) {

1073:                if      (address(interestRateModel)      !=      address(0))
depositToTreasury(updateFloatingDebt());

```

File: InstallmentsRouter.sol

```

33:    weth      =      address(marketWETH_)      !=      address(0)      ?
WETH(payable(address(marketWETH_.asset()))): WETH(payable(0));

```

Recommendation

Add a new function for this type of checks

```

function assembly_notZero(address toCheck) public pure returns(bool success) {

    assembly {

        if iszero(toCheck) {

            let ptr := mload(0x40)

            mstore(ptr, 0xd92e233d00000000000000000000000000000000000000000000000000000000)
// selector for `ZeroAddress()`

            revert(ptr, 0x4)

        }

    }

    return true;
}

```

```
}
```

Status

Acknowledged.

[G-02] Market.sol - Using bools for storage incurs overhead**Description**

Use uint8 = 1 and uint8 = 2 for true/false to avoid a Gwarmaccess (100 gas), and to avoid Gsset (20000 gas) when changing from 'false' to 'true', after having been 'true' in the past.

Instances:

```
File: Market.sol
```

```
94:    bool public isFrozen;
```

```
...
```

Recommendation

Use uint8 = 1 and uint8 = 2 for true/false

Status

Acknowledged.

[G-03] InstallmentsRouter.sol - `'++i'` costs less gas than `'i++'`, especially when it's used in `'for'`-loops (`'--i'/'i--'` too)

Description

`'++i'` costs less gas than `'i++'`, especially when it's used in `'for'`-loops (`'--i'/'i--'` too)

Instances:

File: InstallmentsRouter.sol

```
63:     for (uint256 i = 0; i < amounts.length; i++) {

100:     for (uint256 i = 0; i < amounts.length; i++) totalAmount += amounts[i];

120:     for (uint256 i = 0; i < amounts.length; i++) totalAmount += amounts[i];
```

Status

Resolved.

Centralisation

The Exactly project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

#Hashlock.

Hashlock Pty Ltd

Conclusion

After Hashlocks analysis, the Exactly project seems to have a sound and well tested code base. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au

#Hashlock.



#Hashlock.

Hashlock Pty Ltd